Round Robin and Weighted Round Robin

take turns, and give bigger servers bigger turns

Watch the 4-min video

In one line: round robin sends each new request to the next server in turn, and weighted round robin gives bigger servers proportionally more turns; both are cheap and fair when requests cost about the same, and both go wrong when request costs vary or connections live long, because they count requests, not work.

1 Taking turns

The simplest way to share work is to take turns: server 1, server 2, server 3, then back to 1. That's **round robin**, and it's the default in many load balancers (NGINX, HAProxy, AWS ALB). The balancer keeps a **pointer** into the list of healthy backends; each new request goes to the next one, and the pointer wraps around at the end.

It needs no measurements and no knowledge of the servers, so it costs almost nothing. With identical servers and similar requests, each server gets an equal share.

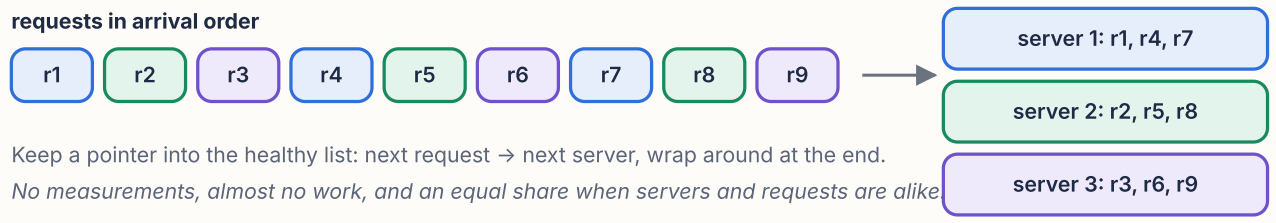


Figure 1. Round robin with three servers: nine requests, three each, in strict rotation.

2 Weighted round robin: unequal servers

Servers often differ: an old instance type and a new one, or a mix of sizes. **Weighted round robin** gives each server a **weight** proportional to its capacity. A server with weight 2 gets two requests for every one sent to a server with weight 1.

A naive version sends the weight-2 server two requests in a row, a little burst. **Smooth weighted round robin**, the algorithm in NGINX, interleaves them so the big server's turns are spread out (C, A, C, B rather than C, C, A, B).

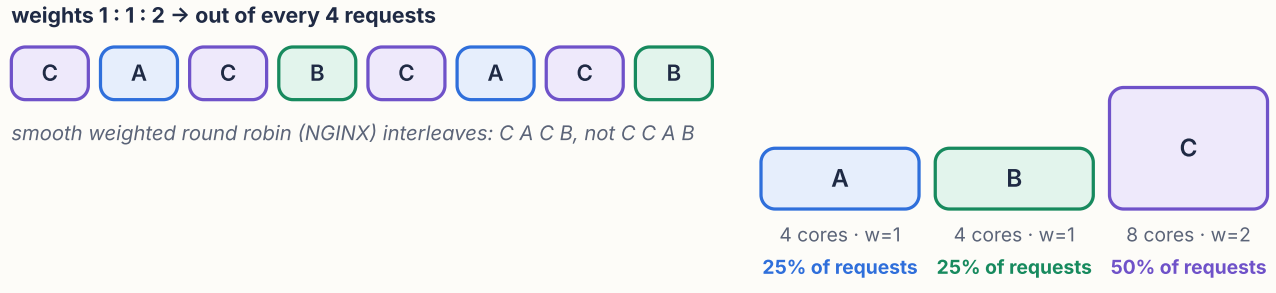


Figure 2. Two 4-core servers with weight 1 and one 8-core server with weight 2: the big one gets half the requests, so CPU stays roughly even.

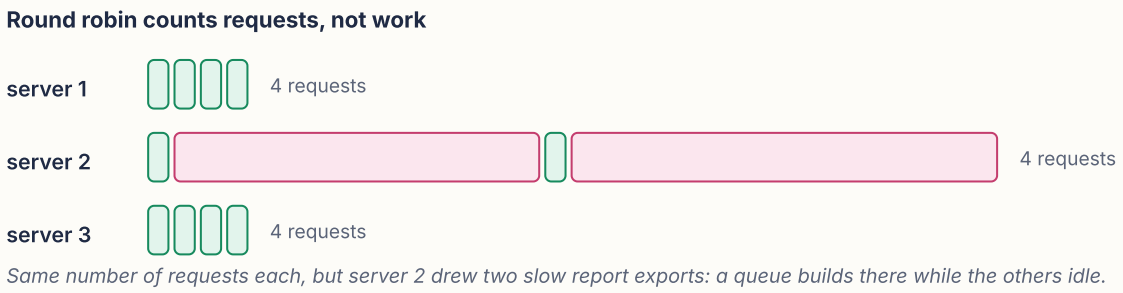
WORKED EXAMPLE: A MIXED POOL

The pool has two 4-core instances and one 8-core instance. Give the small ones weight 1 and the large one weight 2. Out of every 4 requests, the large instance gets 2 and each small one gets 1, so each core sees about the same load. At 1,200 requests per second: 600 to the big server, 300 to each small one.

Weights are also handy for **canary releases**: give the new version weight 1 and the old fleet weight 19, and about 5% of traffic tries the new code (lesson 13 goes further).

3 Where round robin breaks

It counts requests, not work. If most requests take 10 milliseconds but some take 10 seconds (a report export, an LLM answer), one server can draw several slow ones in a row. A queue builds there while its neighbours sit idle, and round robin keeps sending it its fair share anyway.



Same number of requests each, but server 2 drew two slow report exports: a queue builds there while the others idle.

Figure 3. Equal counts, unequal work: the server that drew the slow requests falls behind.

Other traps:

- **Long-lived connections.** WebSockets and gRPC streams are balanced once, when they open. After a scale-up, the old servers keep their connections and the new ones stay nearly empty.
- **Static weights are blind.** A server that is slow because of garbage collection, a bad disk or a noisy neighbour keeps getting its full share.
- **Many balancer nodes.** Each node runs its own rotation; combined, the pattern any one server sees can be lumpy rather than smooth.

4 When to use which

Situation	Choose
Similar servers, short requests of similar cost (typical stateless web APIs)	**Round robin**
Mixed instance sizes, or a gradual rollout / canary	**Weighted round robin**
Request cost varies a lot, or long-lived streams	**Least connections** or latency-aware (next lesson)

COMMON MISTAKES

- Keeping round robin for an endpoint mix that includes slow exports or AI calls, then blaming the servers for high p99 latency.
- Setting weights once and forgetting them after the instance types change.
- Scaling out WebSocket servers and expecting existing connections to move to the new ones.

5 Key terms

Round robin: next request to the next server, wrapping around

Smooth weighted RR: interleaves turns so big servers get no bursts

Long-lived connection: WebSocket or gRPC stream, balanced only once

Weight: a number proportional to a server's capacity

Canary release: a small share of traffic to a new version

Load-aware algorithm: chooses by current load, e.g. least connections

6 Interview questions

Q1. How does round robin work? BASIC

The balancer keeps a pointer into the list of healthy backends and sends each new request to the next one, wrapping around at the end. It needs no measurements, so it is cheap and gives equal shares when servers and requests are alike.

Q2. When does round robin distribute load poorly? BASIC

When request costs vary widely, when connections are long-lived, or when servers differ in capacity or health, because it counts requests, not actual load.

Q3. How does weighted round robin help? BASIC

It gives larger servers proportionally more requests, for example twice the traffic to an instance with twice the cores. Smooth weighted round robin interleaves the turns to avoid bursts.

Q4. You scale a WebSocket fleet from 4 to 8 servers, but the new ones get almost no load. Why, and what do you do? SCENARIO

Existing connections were balanced when they opened and stay put. Use least connections for new connections, and gradually rebalance by asking clients to reconnect (connection max age, or closing a share of connections on the busy servers).

Q5. How would you send 5% of traffic to a new version with round robin? SCENARIO

Weighted round robin: put the new version in the pool with weight 1 against weight 19 for the old fleet (or use a balancer feature such as weighted target groups), watch errors and latency, then raise the weight.

Q6. Your p99 latency is high, yet CPU is low on most servers. The balancer uses round robin. What's going on? TRICKY

Probably uneven work: a few slow requests queue on whichever servers drew them, while others idle. Average CPU hides it. Switch to least connections or power of two choices, or move slow work to an async queue.

Next: SD-G-004 · Least Connections and Least Response Time backendroadmap.com